

Relatório trabalho 2 Big Data

Alunos: Guilherme Portas, Allan Samuel e Mariana Magalhães

Para essa atividade, decidimos seguir com a simulação de Pipeline para ingestão e processamento de dados, onde devemos implementar um sistema de Pipeline completo de ingestão, processamento, armazenamento e visualização de dados. Utilizaremos o Dataset de qualidade do ar disponibilizado no Github. Decidimos seguir com a abordagem B, que utiliza o Airflow como Orquestrador do pipeline para organizar, coordenar e monitorar a execução do Pipeline de dados, excelente para organizar Pipelines complexos, facilidade para a observação da execução, ideal para processos batch e ETL (ETL (extract, transform, load) é um processo de integração de dados que combina, limpa e organiza dados de diferentes fontes em um único repositório de dados) e ótimo para definir regras de dependência claras.

Para começar o projeto, primeiramente executamos todos os aplicativos necessários individualmente utilizando o Docker para criar máquinas virtuais chamadas de contêineres. Executamos o Airflow, para organizar o Pipeline, o Kafka, para armazenar os dados lidos do Dataset em um tópico, o Celery, junto do Redis e o RabbitMQ, que processa os dados recebidos, os envia ao Redis e armazena os dados brutos processados no MinIO, armazena os dados em cache em tempo real, e identifica alertas críticos, respectivamente, o MinIO, que mantém os dados históricos, funcionando como um Data Lake, e por fim, o Streamlit, que lê os dados armazenados do Redis e exibe o status dos sensores em tempo real.

Para o código do Pipeline, criamos um arquivo em Python na pasta “dags” do Airflow, contendo uma cópia ligeiramente modificada do exemplo de DAG do Airflow, um modelo que encapsula tudo o que é necessário para a execução do fluxo de trabalho, disponibilizado no Github do professor. Foi escrito também um arquivo Dockerfile simples contendo a instalação dos programas necessários para execução do Pipeline na imagem do contêiner. São eles: “kafka-python”, “redis”, “pandas”, “pika” e “boto3”.

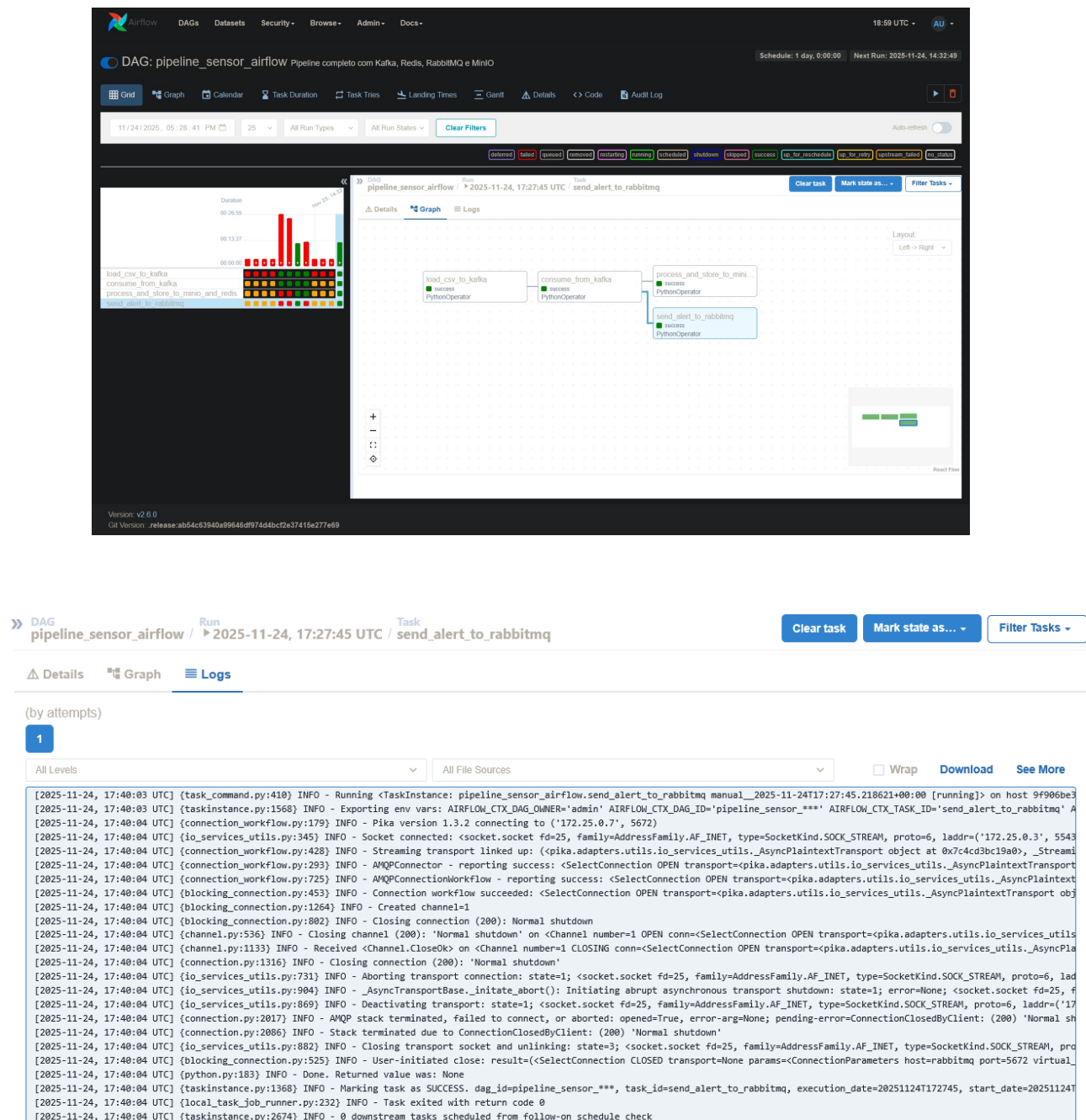
Nossa primeira tentativa de execução não foi bem-sucedida, visto que os ambientes não estavam interligados a partir de uma rede, impossibilitando eles de “conversarem” entre si, como se fosse uma ponte entre os contêineres. Para isso utilizaremos a rede “airflow-net”, na qual definimos a utilização em todos os ambientes na qual executaremos. Após isso, os aplicativos conseguiram se comunicar entre si, o que nos permitiu continuar com a execução do Pipeline. Tivemos outros problemas de comunicação entre o código e o aplicativo necessário executar, por exemplo, o MinIO e o Kafka, então tivemos que ir testando as bibliotecas de um a um para solucionar esses obstáculos.

Após as devidas correções, conseguimos executar o Pipeline completo com todos os aplicativos funcionando corretamente e o código enviando com sucesso uma notificação para o RabbitMQ. Porém, ficou faltando o Streamlit. Nesse aplicativo, nosso objetivo é de criar um código que sirva como um Dashboard, mostrando os dados adquiridos do Redis em tempo real e mostrando quais dados demonstram altos níveis na quantidade de gases dióxido de carbono e monóxido de carbono, nas quais definimos um aviso para valores maiores que 160 para NO₂ e 6 para CO. Esses valores, apesar de serem ainda de médio risco, são valores ótimos para demonstração do sistema de aviso do processamento de dados, com a maioria dos valores lidos sendo abaixo desse limite.

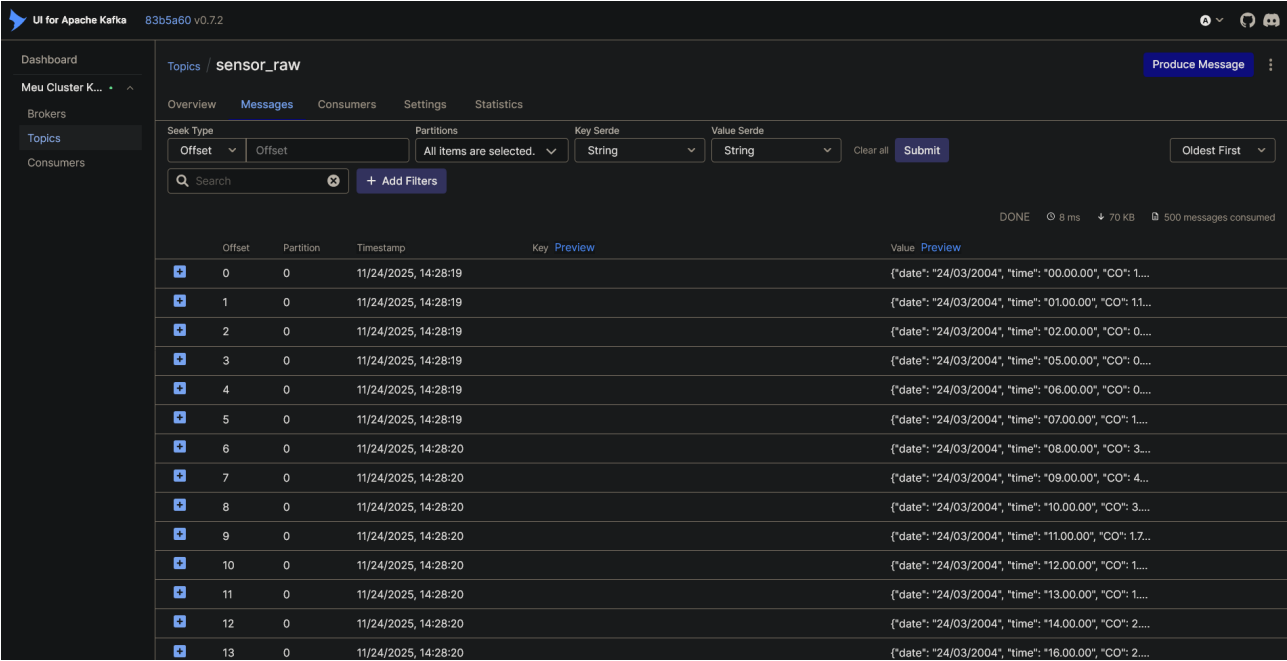
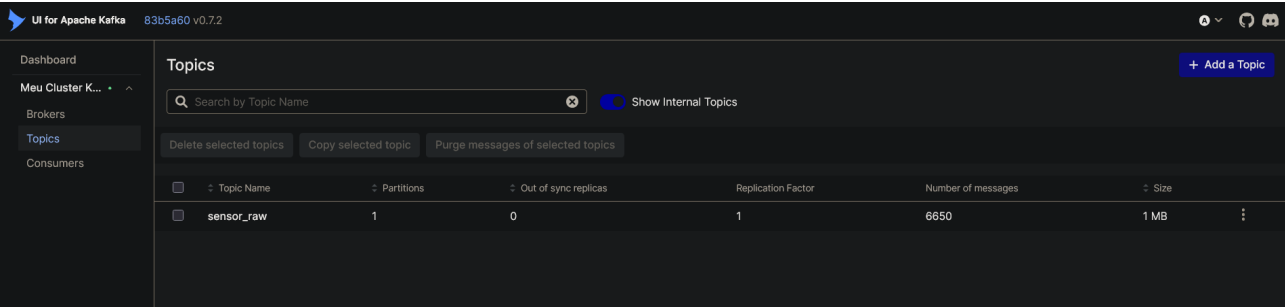
Vale notar também que o Redis possui um limite de 200 chaves, o que acabou também limitando a quantidade de dados que o Streamlit consegue mostrar.

Para o final dessa atividade, criamos também um arquivo Shell contendo a execução e configuração dos ambientes do Docker para facilitar a execução em outras máquinas. Outro ponto importante de mencionar, é que apesar de o Pipeline ter sido desenvolvido para ser executado em paralelo para processamento em tempo real, o código não é executado em paralelo, o que aumentou o tempo de execução e acabou saindo de um dos objetivos do trabalho.

Abaixo, segue algumas imagens obtidas da execução dos ambientes em nossas máquinas.

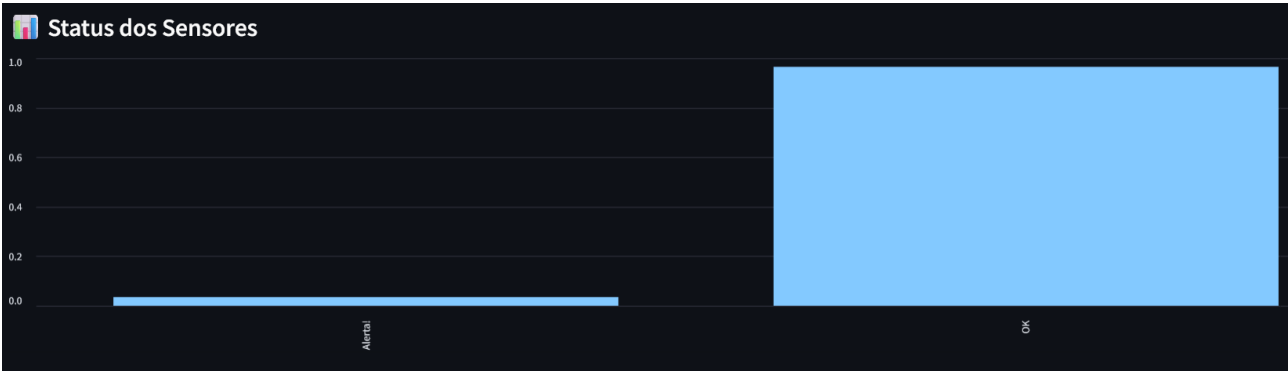


Execução do Pipeline do Airflow, mostrando que todas as tarefas foram executadas com sucesso e mostrando a execução do envio do alerta ao RabbitMQ.

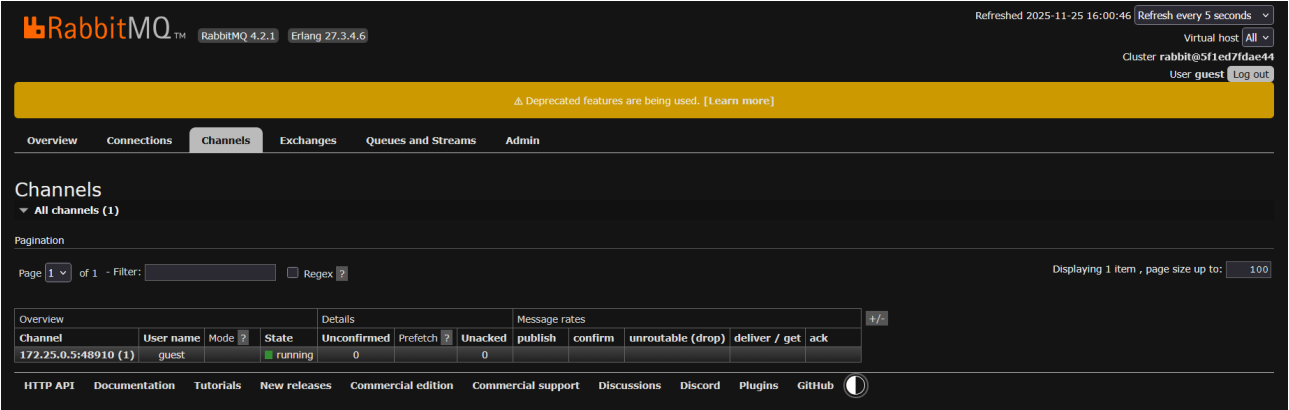
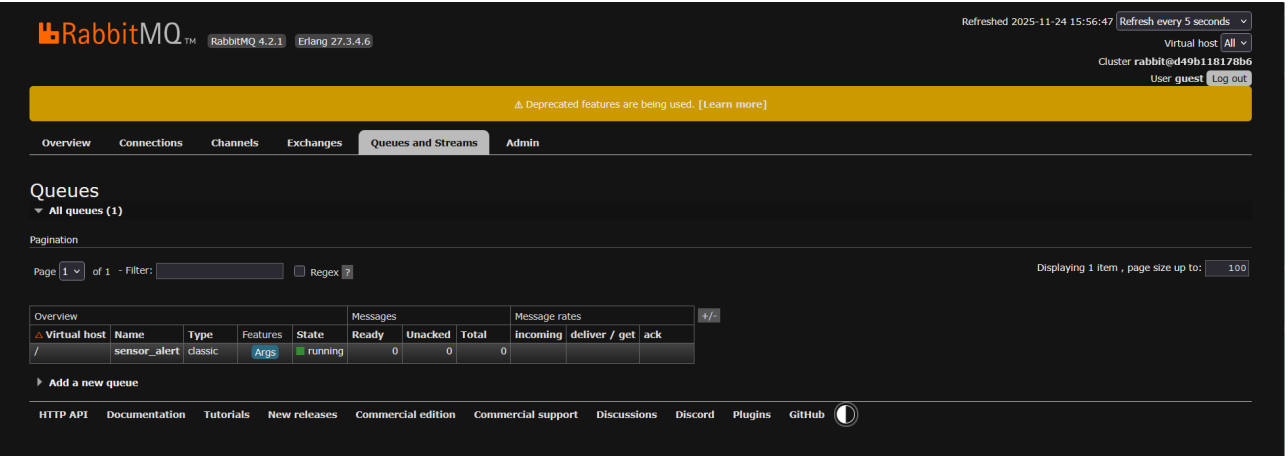


O Kafka obteve com sucesso todos os dados existentes no Dataframe da qualidade do ar.





O Streamlit demonstrando o seu Dashboard com os dados em uma tabela, o status de cada um e comparando a proporção entre dados que necessitam ou não de atenção.



O RabbitMQ obteve uma fila para o sensor de alerta e quando acionado, mostra o canal que está sendo executado pelo aviso enviado pelo Airflow.